ASSIGNMENT 2.2

Distributed Agent Messaging over TCP Sockets

Student	[Student Name]	Roll No	[Roll No]
Course	21CSE525T - Agent Technology	Project	Placement Preparation Agent

Unit 2 - Two agents in two processes. Files: assignments/a2_2/agent_b_server.py and agent_a_client.py

1. Objective

Make two agents exchange structured messages over a network, with one agent reasoning on data the other cannot see, and with the client reacting to the reply.

2. Roles

- **Agent-A (client), Student Prep Agent:** holds the student profile and starts every conversation.
- **Agent-B (server), Placement Cell Agent:** holds a private company database and answers requests. It never starts a conversation.
- Each agent runs in its own OS process and they share no memory; they communicate only through messages.

3. Protocol

TCP is used because a report must arrive complete and in order. TCP is a byte stream, so each JSON message is preceded by a 4-byte big-endian length header (struct.pack('>I')). The receiver reads exactly that many bytes, so messages never merge or split.

Eight intents are used: HANDSHAKE and HANDSHAKE_ACK (protocol check), SYNC_STATE and ELIGIBILITY_REPORT (profile and evaluation), REQUEST_ACTION and ACTION_DIRECTIVE (remedial action), TERMINATE and TERMINATE_ACK (close). An unknown intent gets an ERROR reply.

4. Implementation

```
def recv_exact(sock, n):
    buf = b""
    while len(buf) &lt; n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            return None
        buf += chunk
    return buf

def recv_msg(sock):
    """Read 4-byte header then exactly that many bytes; deserialize JSON."""
    hdr = recv_exact(sock, 4)
    if not hdr:
        return None
    (length,) = struct.unpack("&gt;I", hdr)
    body = recv_exact(sock, length)
    if body is None:
        return None
    return json.loads(body.decode("utf-8"))

def evaluate_eligibility(profile):
    eligible, blocked = [], []
    for c in COMPANY_DB:
        reasons = []
        if profile["cgpa"] &lt; c["cgpa"]:
            reasons.append(f"CGPA {profile['cgpa']}&lt;{c['cgpa']}")
        if profile["coding_score"] &lt; c["coding"]:
            reasons.append(f"coding {profile['coding_score']}&lt;{c['coding']}")
        if profile["aptitude_score"] &lt; c["aptitude"]:
            reasons.append(f"aptitude {profile['aptitude_score']}&lt;{c['aptitude']}")
        if reasons:
            blocked.append({"company": c["name"], "gaps": reasons})
        else:
            eligible.append({"company": c["name"], "package_lpa": c["package"]})
```

```
    eligible.sort(key=lambda x: -x["package_lpa"])
    return eligible, blocked

def main():
    print("=" * 78)
    print(" AGENT-A :: STUDENT PREP AGENT (socket client)  [Assignment 2.2]")
    print("=" * 78)

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        log(f"TCP channel established -&gt; {HOST}:{PORT}")

        # ---- 1. HANDSHAKE ----
        send_msg(s, {"intent": "HANDSHAKE", "sender": "AGENT-A", "protocol": "PPA/1.0"})
        log("TX intent=HANDSHAKE")
        r = recv_msg(s)
        log(f"RX intent={r['intent']} status={r.get('status')}")
        time.sleep(0.4)

        # ---- 2. SERIALIZED STATE TRANSFER ----
        payload = {"intent": "SYNC_STATE", "sender": "AGENT-A", "payload": AGENT_A_STATE}
        send_msg(s, payload)
        log(f"TX intent=SYNC_STATE  ({len(json.dumps(payload))} bytes serialized)")
        r = recv_msg(s)
        log(f"RX intent={r['intent']}")

        p = r["payload"]
        print("\n   ---- AGENT-B ELIGIBILITY REPORT ----")
        for e in p["eligible"]:
            print(f"      ELIGIBLE : {e['company']:&lt;12} {e['package_lpa']} LPA")
        for b in p["blocked"]:
            print(f"      BLOCKED  : {b['company']:&lt;12} gaps -&gt; {', '.join(b['gaps'])}")
        print(f"      ADVICE   : {p['advice']}\n")
        time.sleep(0.4)

        # ---- 3. AUTONOMOUS REACTION to remote payload ----
        if p["blocked"]:
            log("AUTONOMOUS DECISION: gaps detected -&gt; requesting remedial action")
            send_msg(s, {"intent": "REQUEST_ACTION", "sender": "AGENT-A"})
            r = recv_msg(s)
            d = r["payload"]
            log(f"RX intent={r['intent']} -&gt; {d['directive']} @ {d['slot']}")
            log("PLAN UPDATED: mock interview auto-added to local study calendar")
        time.sleep(0.4)

        # ---- 4. TERMINATE ----
        send_msg(s, {"intent": "TERMINATE", "sender": "AGENT-A"})
        r = recv_msg(s)
        log(f"RX intent={r['intent']} -- session closed cleanly")

    print("=" * 78)
```

5. Result

The server evaluated the profile against 6 companies: 5 eligible and 1 blocked. Agent-A read the report and, because a gap existed, sent REQUEST_ACTION on its own, and received a mock-interview slot. The session then closed cleanly with TERMINATE_ACK.

Execution output (log file)

Captured from **logs/2.2_server_agent_b.log** (server: python3 -u agent_b_server.py). Timestamps and some values differ between runs; this is the recorded run.

```
# Activity 2.2 SERVER (Agent-B, Placement Cell)
# command: python3 -u agent_b_server.py

==============================================================================
 AGENT-B :: PLACEMENT CELL AGENT (socket server)  [Assignment 2.2]
==============================================================================
[10:40:36] [AGENT-B/SERVER] listening on 127.0.0.1:50607 ...
[10:40:38] [AGENT-B/SERVER] connection accepted from ('127.0.0.1', 55919)
[10:40:38] [AGENT-B/SERVER] ingested payload intent='HANDSHAKE' from 'AGENT-A'
[10:40:38] [AGENT-B/SERVER] dispatched reply intent='HANDSHAKE_ACK'
[10:40:39] [AGENT-B/SERVER] ingested payload intent='SYNC_STATE' from 'AGENT-A'
[10:40:39] [AGENT-B/SERVER] reasoned -&gt; 5 eligible / 1 blocked
[10:40:39] [AGENT-B/SERVER] dispatched reply intent='ELIGIBILITY_REPORT'
[10:40:39] [AGENT-B/SERVER] ingested payload intent='REQUEST_ACTION' from 'AGENT-A'
```

```
[10:40:39] [AGENT-B/SERVER] dispatched reply intent='ACTION_DIRECTIVE'
[10:40:40] [AGENT-B/SERVER] ingested payload intent='TERMINATE' from 'AGENT-A'
[10:40:40] [AGENT-B/SERVER] dispatched reply intent='TERMINATE_ACK'
[10:40:40] [AGENT-B/SERVER] connection with ('127.0.0.1', 55919) closed
```

Execution output (log file)

Captured from **logs/2.2_client_agent_a.log** (client: python3 agent_a_client.py). Timestamps and some values differ between runs; this is the recorded run.

```
# Activity 2.2 CLIENT (Agent-A, Student Prep)
# command: python3 agent_a_client.py
# exit code: 0

================================================================================
 AGENT-A :: STUDENT PREP AGENT (socket client)  [Assignment 2.2]
================================================================================
[10:40:38] [AGENT-A/CLIENT] TCP channel established -&gt; 127.0.0.1:50607
[10:40:38] [AGENT-A/CLIENT] TX intent=HANDSHAKE
[10:40:38] [AGENT-A/CLIENT] RX intent=HANDSHAKE_ACK status=OK
[10:40:39] [AGENT-A/CLIENT] TX intent=SYNC_STATE  (326 bytes serialized)
[10:40:39] [AGENT-A/CLIENT] RX intent=ELIGIBILITY_REPORT

   ---- AGENT-B ELIGIBILITY REPORT ----
      ELIGIBLE : Freshworks   22.0 LPA
      ELIGIBLE : Zoho         18.0 LPA
      ELIGIBLE : TCS Digital  7.0 LPA
      ELIGIBLE : Infosys      6.5 LPA
      ELIGIBLE : Wipro        5.5 LPA
      BLOCKED  : Amazon       gaps -&gt; CGPA 7.6&lt;8.0, coding 79&lt;85, aptitude 74&lt;80
      ADVICE   : Focus on: CGPA, aptitude, coding

[10:40:39] [AGENT-A/CLIENT] AUTONOMOUS DECISION: gaps detected -&gt; requesting remedial action
[10:40:39] [AGENT-A/CLIENT] RX intent=ACTION_DIRECTIVE -&gt; SCHEDULE_MOCK_INTERVIEW @ 2026-09-12 10:00 IST
[10:40:39] [AGENT-A/CLIENT] PLAN UPDATED: mock interview auto-added to local study calendar
[10:40:40] [AGENT-A/CLIENT] RX intent=TERMINATE_ACK -- session closed cleanly
================================================================================
```

6. Viva points

Why TCP and not UDP? Reports must be complete and ordered; UDP can drop or reorder.

Why a length header? Without it, a message can arrive split across reads or joined to the next one.

Why is this multi-agent? Two processes, separate knowledge, message passing, and autonomous reaction by Agent-A.

Weakness: no authentication. Any process could connect to the port. Unit 5 addresses this.

7. Conclusion

The two agents complete a full negotiation of state and action over a real socket, with the length-prefixed framing guaranteeing message boundaries.